Space-Vector Pulse Width Modulation (SVPWM)

KSKelvin Kelvin Leung
Created on : 4-5-2026
Latest Update : 4-7-2026

Basic Operation of SVPWM

Introduction : Space-Vector PWM (SVPWM)

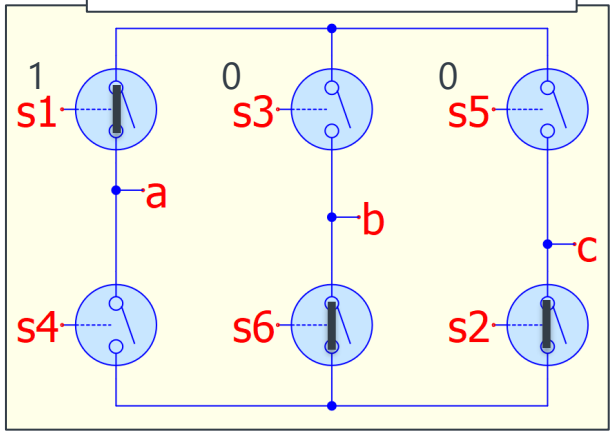
- Space-Vector PWM (SVPWM)
  - Two-level space-vector pulse width modulation (SVPWM) is a sophisticated control algorithm used to drive three-phase AC motors and power converters. Unlike standard Sinusoidal PWM (SPWM), which treats each phase independently, SVPWM treats the three-phase inverter as a single unit to generate a rotating voltage vector

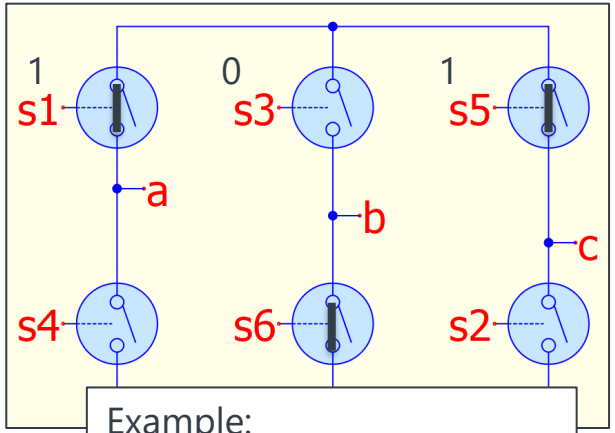
- Reference
  - Altair PSIM symbol : **[Space Vector PWM (2-level)]** – select symbol and press F1 for Help

#1 : Physical Framework

- Physical Framework : The Three Phase Inverter
  - Two-level inverter has three “legs” (a,b,c), each with two switches
    - State 1 : Upper switch is ON and lower switch is OFF (Phase connected to Vdc)
    - State 2: Upper switch is OFF and lower switch is ON (Phase connected to GND)
    - Upper and lower switch must operate complementarily and therefore, total 2^3=8 switching combination
  - Switch Representation : (s1,s3,s5) or (a,b,c)
  - V0 (000), V1 (100), V2 (110), V3 (010), V4 (011), V5 (001), V6 (101), V7 (111)
    - Six Active Vectors (V1 through V6) : Form the perimeter of a hexagon
    - Two Zero Vectors (V0 and V7) : Resulting in zero net voltage across the load (either all 1s or all 0s)

Example:
V1 (100) : s1=1, s3=0, s5=0





Example:
V6 (101) : s1=1, s3=0, s5=1

#2 : Clarke Transformation and Target Reference Vector

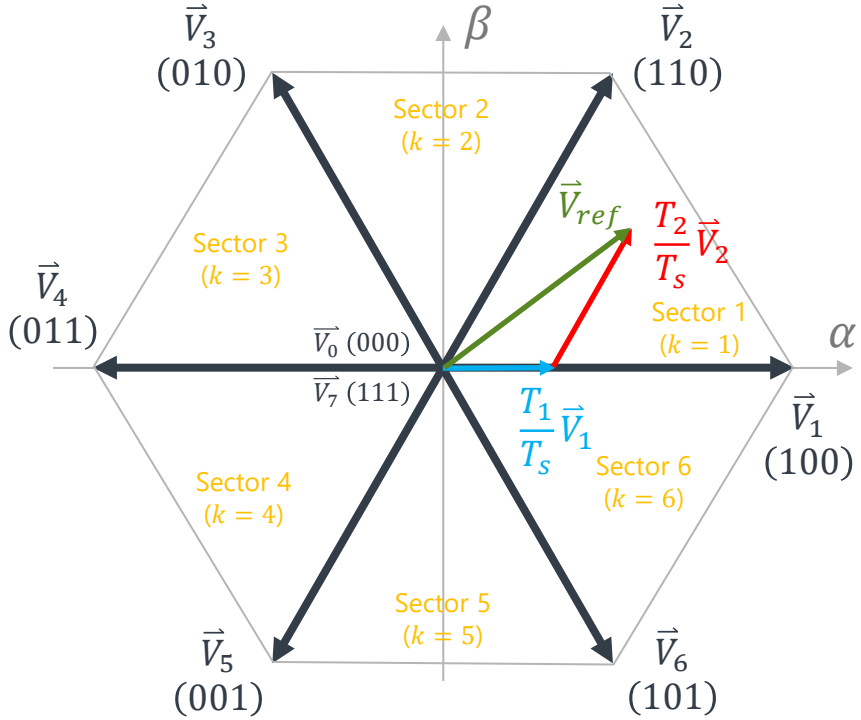
- The Clarke Transformation
  - To simplify the control, the three-phase voltage (V_a, V_b, V_c) are mathematically transformed into a two-dimensional stationary reference frame (α, β)
  - Formula of amplitude invariant transformation

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \frac{2}{3} \cdot \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix}$$

- Target Reference Vector
  - Target Reference Vector $(\vec{V}_{ref} = \alpha + j\beta)$ represents the desired magnitude and angle of the output voltage in (α, β) frame
    - $|V_{ref}| = \sqrt{\alpha^2 + \beta^2}$
    - $\angle V_{ref} = \theta_{ref} = \tan^{-1}\left(\frac{\beta}{\alpha}\right)$
      - Programming code : $\theta_{ref} = atan2(-\beta, -\alpha) + \pi$, it will output angle between $[0, 2\pi)$

#3: Sector Identification and Vector Synthesis

- Sector identification and Vector Synthesis
  - The SVPWM algorithm determines which **sector** of the hexagon V_{ref} is currently in



Since target reference voltage ($\vec{V}_{ref}$) usually fall between two sectors, the inverter cannot produce it directly. Instead, it "average" the two adjacent active vectors and the zero vector (000) or (111) over a sampling period ($T_s = \frac{1}{f_s}$)

For example, If $\vec{V}_{ref}$ in sector 1 between $\vec{V}_1$ and $\vec{V}_2$
The controller calculates

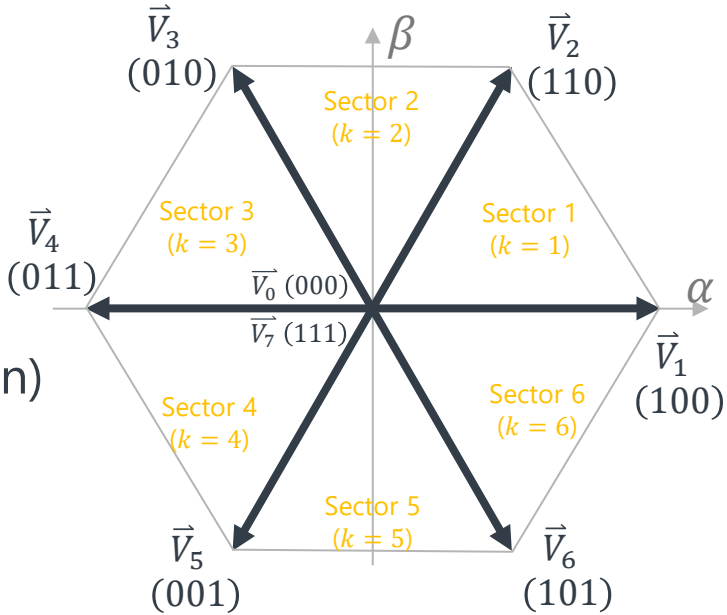
1. T_1 : time spend at $\vec{V}_1$
2. T_2 : time spend at $\vec{V}_2$
3. T_0 : time spend at $\vec{V}_0$ or $\vec{V}_7$

The relationship by volt-second balance

$$V_{ref}T_s = V_1T_1 + V_2T_2 + V_{0|7}T_0$$

#4 : Sector identification (Sector k)

- Sector identification (Sector k)
  - The most direct way to find the sector is to calculate $\angle V_{ref} = \theta_{ref} = \tan^{-1}\left(\frac{\beta}{\alpha}\right)$
    - Sector 1 : $0° \leq \theta_{ref} < 60°$
    - Sector 2 : $60° \leq \theta_{ref} < 120°$
    - Sector 3 : $120° \leq \theta_{ref} < 180°$
    - Sector 4 : $180° \leq \theta_{ref} < 240°$
    - Sector 5 : $240° \leq \theta_{ref} < 300°$
    - Sector 6 : $300° \leq \theta_{ref} < 360°$
    - Programming code : $sector(k) = \text{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) + 1$



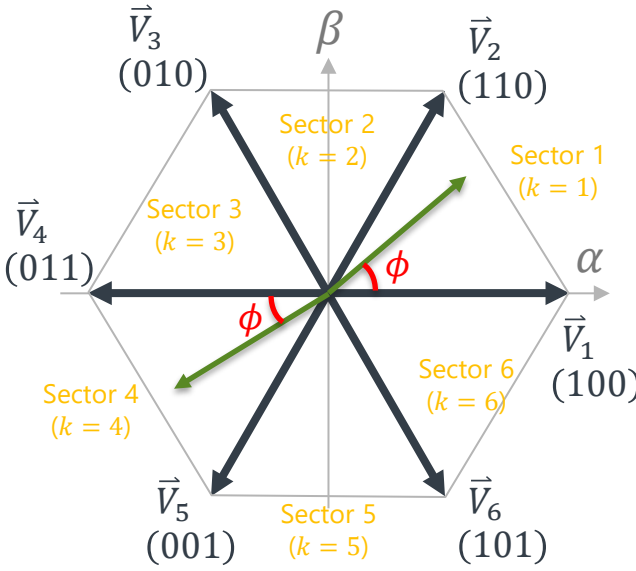
- Basic Formula for Timing calculation (next section)
  - Assume DC link voltage amplitude is V_{dc}
  - Active states are represented by space vectors
    - $\vec{V}_k = \frac{2}{3} V_{dc}\, e^{j\frac{(k-1)\pi}{3}} = \frac{2}{3} V_d \left(\cos\frac{(k-1)\pi}{3} + j\sin\frac{(k-1)\pi}{3}\right)$
  - From volt-second balance
    - $V_{ref} T_s = V_k T_k + V_{k+1} T_{k+1}$

#5 : Timing Calculation (T_k and T_{k+1}) – $\alpha\beta$-axis (stationary frame)

- Timing calculation for T_k and T_{k+1} – $\alpha\beta$-axis (stationary frame)
  - In sector k (where $k = 1,2,3,..6$), timing of two adjacent vectors V_k, V_{k+1} are T_k, T_{k+1}
  - By $V_{ref}T_s = V_kT_k + V_{k+1}T_{k+1}$
  - $V_{ref}T_s = \left(\frac{2}{3}\, V_{dc}\, e^{j\frac{(k-1)\pi}{3}}\right)T_k + \left(\frac{2}{3}\, V_{dc}\, e^{j\frac{k\pi}{3}}\right)T_{k+1}$
  - $(\alpha + j\beta)T_s = \frac{2}{3}\, V_{dc}\left(T_k\, e^{j\frac{(k-1)\pi}{3}} + T_{k+1}\, e^{j\frac{k\pi}{3}}\right)$
  - $(\alpha + j\beta)T_s = \frac{2}{3}\, V_{dc}\left(T_k \cos\frac{(k-1)\pi}{3} + jT_k \sin\frac{(k-1)\pi}{3} + T_{k+1}\cos\frac{k\pi}{3} + jT_{k+1}\sin\frac{k\pi}{3}\right)$
  - Solving above equation for T_k and T_{k+1} (very complicated mathematic process)
    - $\begin{bmatrix} T_k \\ T_{k+1} \end{bmatrix} = \frac{\sqrt{3}\, T_s}{V_{dc}} \begin{bmatrix} \sin\frac{k\pi}{3} & -\cos\frac{k\pi}{3} \\ -\sin\frac{(k-1)\pi}{3} & \cos\frac{(k-1)\pi}{3} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$
    - By $\vec{V}_{ref} = \alpha + j\beta = |V_{ref}|\left(\cos(\theta_{ref}) + j\sin(\theta_{ref})\right)$, θ_{ref} also refer as ωt
    - $\begin{bmatrix} T_k \\ T_{k+1} \end{bmatrix} = \frac{\sqrt{3}\, T_s |V_{ref}|}{V_{dc}} \begin{bmatrix} \sin\frac{k\pi}{3} & -\cos\frac{k\pi}{3} \\ -\sin\frac{(k-1)\pi}{3} & \cos\frac{(k-1)\pi}{3} \end{bmatrix} \begin{bmatrix} \cos(\theta_{ref}) \\ \sin(\theta_{ref}) \end{bmatrix}$
    - Where V_{dc} can set to 1 for nominalization (T_S can also set to 1 if 0-1V sawtooth is used for PWM comparison)
    - And $T_0 = T_s - (T_k + T_{k+1})$

#5 : Timing Calculation (T_k and T_{k+1}) – $\vec{V}_k$-axis (sector bound local frame)

- Timing calculation for T_k and T_{k+1} – $\vec{V}_k$-axis (sector bound local frame)
  - This method uses relative angle within the current sector (0° to 60°), with respect to $\vec{V}_k$
  - By $\begin{bmatrix} T_k \\ T_{k+1} \end{bmatrix} = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \begin{bmatrix} sin\frac{k\pi}{3} & -cos\frac{k\pi}{3} \\ -sin\frac{(k-1)\pi}{3} & cos\frac{(k-1)\pi}{3} \end{bmatrix} \begin{bmatrix} cos(\theta_{ref}) \\ sin(\theta_{ref}) \end{bmatrix}$
  - In sector 1 where k=1
    - $\begin{bmatrix} T_1 \\ T_2 \end{bmatrix} = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \begin{bmatrix} sin\frac{\pi}{3} & -\frac{1}{2} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} cos(\theta_{ref}) \\ sin(\theta_{ref}) \end{bmatrix}$
    - $T_1 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \left(\sin\frac{\pi}{3}\cos(\theta_{ref}) - \frac{1}{2}\sin(\theta_{ref})\right) = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \sin\left(\frac{\pi}{3} - \theta_{ref}\right)$
    - $T_2 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} sin(\theta_{ref})$
    - where $|V_{ref}| = \sqrt{\alpha^2 + \beta^2}$ and $\angle V_{ref} = \theta_{ref} = \tan^{-1}\left(\frac{\beta}{\alpha}\right)$
  - General form for all sector, where ϕ is angle between $\vec{V}_{ref}$ and $\vec{V}_k$
    - $T_k = T_1 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \sin\left(\frac{\pi}{3} - \phi\right)$
    - $T_{k+1} = T_2 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} sin\phi$
    - where $\phi = \theta_{ref} - \text{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) \cdot \frac{\pi}{3}$



#5 : Timing Calculation (T_k and T_{k+1}) – Summary

- Approach #1 – absolute angle reference to $\alpha\beta$-axis (stationary frame)
  - $\begin{bmatrix} T_k \\ T_{k+1} \end{bmatrix} = \frac{\sqrt{3}\,T_s}{V_{dc}} \begin{bmatrix} sin\frac{k\pi}{3} & -cos\frac{k\pi}{3} \\ -sin\frac{(k-1)\pi}{3} & cos\frac{(k-1)\pi}{3} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$ **OR** $\begin{bmatrix} T_k \\ T_{k+1} \end{bmatrix} = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \begin{bmatrix} sin\frac{k\pi}{3} & -cos\frac{k\pi}{3} \\ -sin\frac{(k-1)\pi}{3} & cos\frac{(k-1)\pi}{3} \end{bmatrix} \begin{bmatrix} cos(\theta_{ref}) \\ sin(\theta_{ref}) \end{bmatrix}$
  - General trigonometry
    - Global Perspective: treat the hexagon as a single 360° field
    - Computational Heavy: It requires calculating sin and cosine for the specific angle in every cycle
    - Intuitive: It is very easy to visualize as a vector rotating smoothly in space (Academic/Theoretical modelling)

- Approach #2 – relative angle reference to $\vec{V}_k$-axis (sector bound local frame)
  - $T_k = T_1 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \sin\left(\frac{\pi}{3} - \phi\right)$, $T_{k+1} = T_2 = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} sin\phi$
    - where $\phi = \theta_{ref} - \mathrm{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) \cdot \frac{\pi}{3}$
  - Local geometry
    - Local Perspective: The controller only cares about where the vector is relative to the "start" of the current sector
    - Normalized Logic: You can use the same logic/code for every sector by simply swapping which physical switches correspond to T_1 and T_2
    - For real time control: allows the processor to skip complex math

#6: Implementation with 3-Segment Switching Scheme

- Implementation with 3-Segment Switching Scheme
  - To demonstrate most fundamental implementation of SVPWM with non-symmetric 3-segment switching scheme
  - Since T_k and T_{k+1} is computed, target reference vector can be generated with
  - $V_{ref}\ T_S = V_k\ T_k + V_{k+1}\ T_{k+1} + V_0|V_7\ T_0$
    - where $T_0 = T_s - T_k - T_{k+1}$
  - For example, in sector 1 (k=1), this represent
    - $V_0(000)$ or $V_7(111)$ turn on for T_0 duration
    - $V_1(100)$ turn on for T_k duration
    - $V_2(110)$ turn on for T_{k+1} duration
  - Here is a table summarize turn on duration, sector and switching pattern relationship

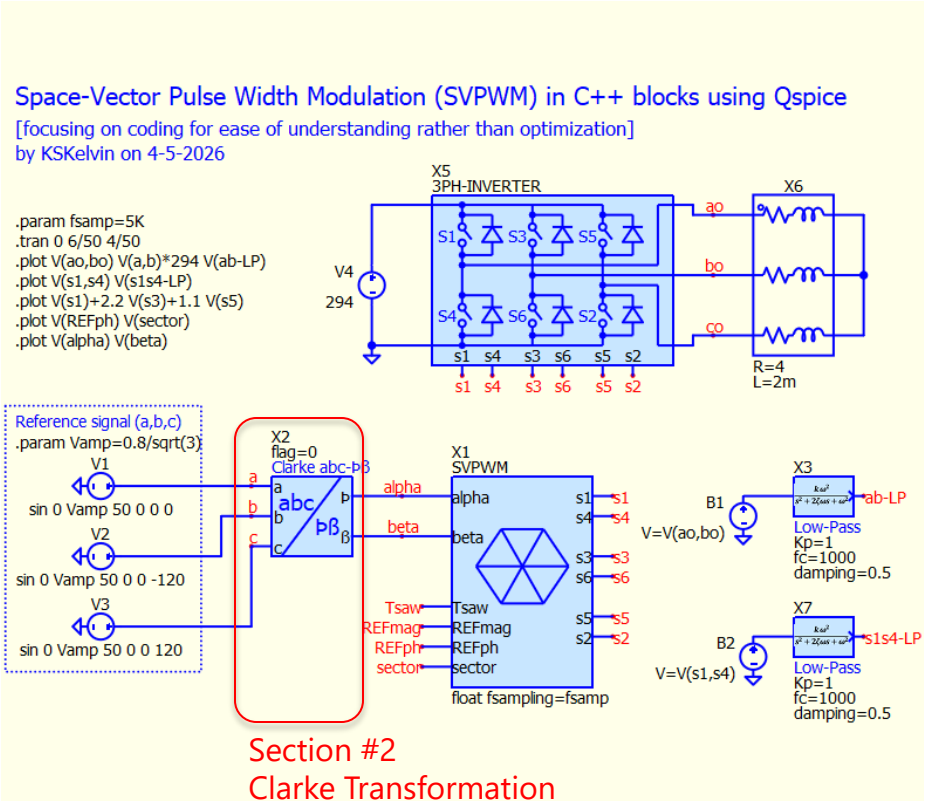
	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
T_0	$V_0(000)$	$V_7(111)$	$V_0(000)$	$V_7(111)$	$V_0(000)$	$V_7(111)$
T_k	$V_1(100)$	$V_2(110)$	$V_3(010)$	$V_4(011)$	$V_5(001)$	$V_6(101)$
T_{k+1}	$V_2(110)$	$V_3(010)$	$V_4(011)$	$V_5(001)$	$V_6(101)$	$V_1(100)$

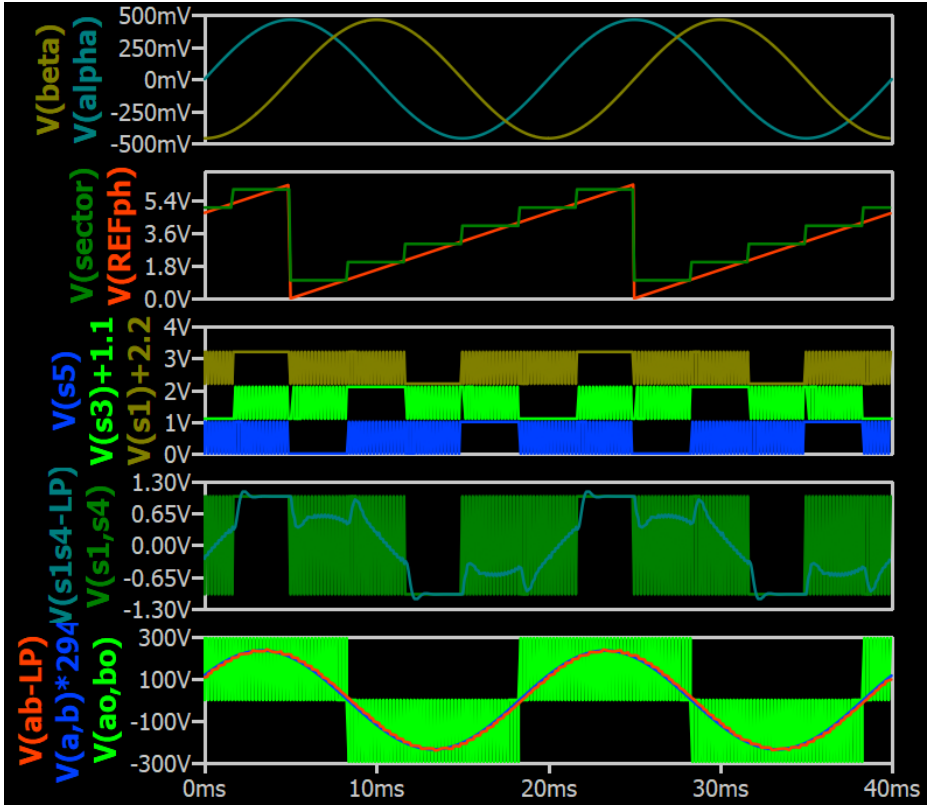
#6: Implementation with 3-Segment Switching Scheme

- 3-Segment Switching Sequence
  - This 3-segment switching sequence follows a specific pattern with only **single-bit changes** to minimize switching actions during transitions
  - Sector 1 : $V_0(000) \rightarrow V_1(100) \rightarrow V_2(110)$
  - Sector 2 : $V_7(111) \rightarrow V_2(110) \rightarrow V_3(010)$
  - Sector 3 : $V_0(000) \rightarrow V_3(010) \rightarrow V_4(011)$
  - Sector 4 : $V_7(111) \rightarrow V_4(011) \rightarrow V_5(001)$
  - Sector 5 : $V_0(000) \rightarrow V_5(001) \rightarrow V_6(101)$
  - Sector 6 : $V_7(111) \rightarrow V_6(101) \rightarrow V_1(100)$
  - As a result, switching sequence are
    - In odd sector ($\boldsymbol{k = 1, 3, 5}$): $\boldsymbol{V_0 \rightarrow V_k \rightarrow V_{k+1}}$
    - In even sector ($\boldsymbol{k = 2, 4, 6}$): $\boldsymbol{V_7 \rightarrow V_k \rightarrow V_{k+1}}$
    - where timing of $\boldsymbol{V_0}$ / $\boldsymbol{V_7}$ is $\boldsymbol{T_0}$, $\boldsymbol{V_k}$ is $\boldsymbol{T_k}$ and $\boldsymbol{V_{k+1}}$ is $\boldsymbol{T_{k+1}}$

#6: Implementation with 3-Segment Switching Scheme

Qspice : \01 Basic Operation (DLL 3-segment)\example.Space-VectorPWM(3-segment).qsch





Custom Symbol in https://github.com/KSKelvin-Github : Symbols-KSKelvin/power electronics - 3phase/function blocks

#6: Implementation with 3-Segment Switching Scheme

Qspice : \01 Basic Operation (DLL 3-segment)\svpwm.cpp

Section #2 : Target Reference Vector

```
// Calculate PWM period from sampling frequency
double Ts=1.0/fsampling;

// Calculate reference voltage vector magnitude and angle
REFmag = hypot(alpha,beta);          // Euclidean norm: sqrt(alpha²
REFph = atan2(-beta,-alpha)+M_PI;    // Phase angle from Cartesian

// Determine which sector the reference vector falls into (6 secto
sector = trunc(REFph/(M_PI/3))+1;

/// Generate triangular sawtooth waveform synchronized with sampli
Tsaw = fmod(t,Ts);    // Modulo operation creates repeating ramp fr

// Calculate duty cycles for adjacent active vectors (Tk and Tk+1)
// Based on alpha-beta (stationary reference frame) components
double Tk      = sqrt(3.0)*Ts*( sin(sector*M_PI/3.0)*alpha - cos(s
double Tkplus1 = sqrt(3.0)*Ts*( -sin((sector-1)*M_PI/3.0)*alpha +
// Zero vector on-time (remaining time in switching period)
double T0      = Ts - (Tk + Tkplus1);
```

Section #4: Sector identification

Section #5: Timing calculation

Section #6 : 3-segment Switching Sequence

```
// Three-segment SVPWM pattern generation for each sector
if(sector == 1){
   // Sector 1 switching sequence:
   //  V0(000) -> V1(100) -> V2(110)
   if(Tsaw < T0)            {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0+Tk)) {s1=true ;s3=false;s5=false;}
   else                     {s1=true ;s3=true ;s5=false;}
}
if(sector == 2){
   // Sector 2 switching sequence:
   //  V7(111) -> V2(110) -> V3(010)
   if(Tsaw < T0)            {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0+Tk)) {s1=true ;s3=true ;s5=false;}
   else                     {s1=false;s3=true ;s5=false;}
}
if(sector == 3){
   // Sector 3 switching sequence:
   //  V0(000) -> V3(010) -> V4(011)
   if(Tsaw < T0)            {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0+Tk)) {s1=false;s3=true ;s5=false;}
   else                     {s1=false;s3=true ;s5=true ;}
}
if(sector == 4){
   // Sector 4 switching sequence:
   //  V7(111) -> V4(011) -> V5(001)
   if(Tsaw < T0)            {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0+Tk)) {s1=false;s3=true ;s5=true ;}
   else                     {s1=false;s3=false;s5=true ;}
}
if(sector == 5){
   // Sector 5 switching sequence:
   //  V0(000) -> V5(001) -> V6(101)
   if(Tsaw < T0)            {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0+Tk)) {s1=false;s3=false;s5=true ;}
   else                     {s1=true ;s3=false;s5=true ;}
}
if(sector == 6){
   // Sector 6 switching sequence:
   //  V7(111) -> V6(101) -> V1(100)
   if(Tsaw < T0)            {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0+Tk)) {s1=true ;s3=false;s5=true ;}
   else                     {s1=true ;s3=false;s5=false;}
}
```

Switching Pattern
7-segment Algorithm

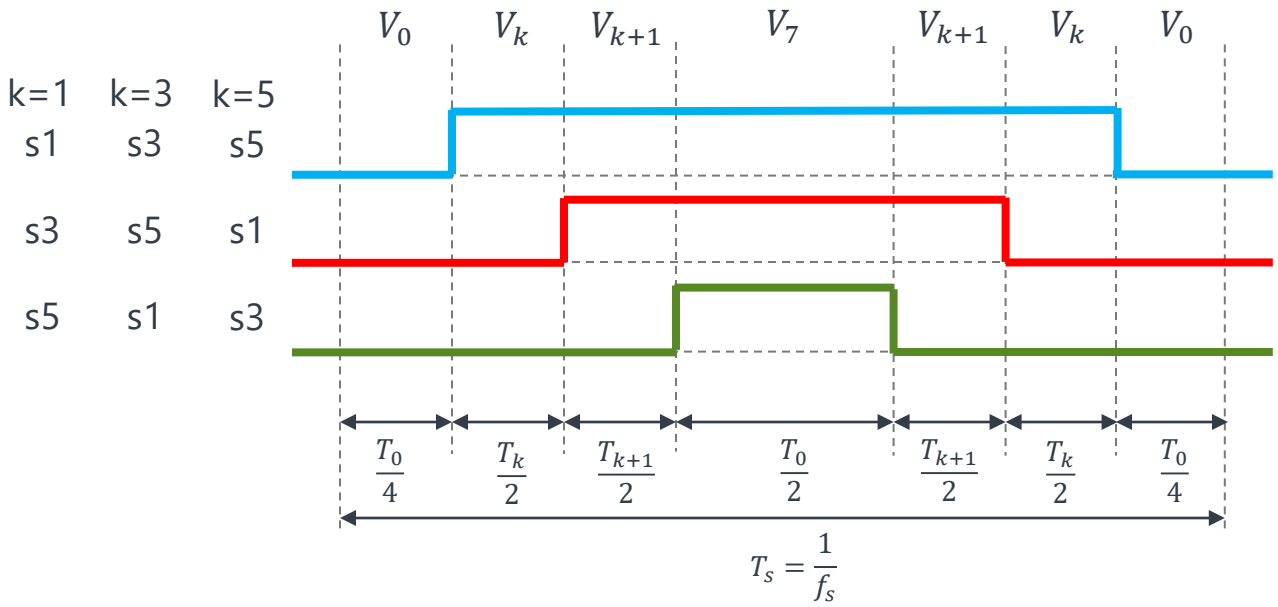
#7 : Switching Sequence – 7 segment scheme

- Switching Sequence – 7 segment scheme
  - To minimize harmonic distortion and switching losses, the algorithm follows a specific sequence with only **Single-Bit Change** (i.e. 100 → 110). A common method is the Seven-Segment Symmetrical Sequence
  - For example, Sector 1, the switches would transition as:
    - $\boldsymbol{V_0}\ (\mathbf{000}) \rightarrow \boldsymbol{V_1}\ (\mathbf{100}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_7}\ (\mathbf{111}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_1}\ (\mathbf{100}) \rightarrow \boldsymbol{V_0}\ (\mathbf{000})$
      where timing of V_0/V_7 is T_0, V_1 is T_1 and V_2 is T_2
  - In Sector 2, the switches would transition as:
    - $\boldsymbol{V_0}\ (\mathbf{000}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_7}\ (\mathbf{111}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_0}\ (\mathbf{000})$
    - However, this would cause Double-Bit Change in certain transition, therefore, swap V_0 and V_7
    - $\boldsymbol{V_7}\ (\mathbf{111}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_0}\ (\mathbf{000}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_7}\ (\mathbf{111})$
    - However, from Sector 1 to 2, it would transit from V_0 (000) to V_7 (111), therefore, shift the sequence
    - $\boldsymbol{V_0}\ (\mathbf{000}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_7}\ (\mathbf{111}) \rightarrow \boldsymbol{V_2}\ (\mathbf{110}) \rightarrow \boldsymbol{V_3}\ (\mathbf{010}) \rightarrow \boldsymbol{V_0}\ (\mathbf{000})$
  - As a result, optimized 7-segment switching sequence are
    - In odd sector ($\boldsymbol{k} = \mathbf{1, 3, 5}$): $\boldsymbol{V_0} \rightarrow \boldsymbol{V_k} \rightarrow \boldsymbol{V_{k+1}} \rightarrow \boldsymbol{V_7} \rightarrow \boldsymbol{V_{k+1}} \rightarrow \boldsymbol{V_k} \rightarrow \boldsymbol{V_0}$
    - In even sector ($\boldsymbol{k} = \mathbf{2, 4, 6}$): $\boldsymbol{V_0} \rightarrow \boldsymbol{V_{k+1}} \rightarrow \boldsymbol{V_k} \rightarrow \boldsymbol{V_7} \rightarrow \boldsymbol{V_k} \rightarrow \boldsymbol{V_{k+1}} \rightarrow \boldsymbol{V_0}$
    - where timing of $\boldsymbol{V_0}$ / $\boldsymbol{V_7}$ is T_0 , $\boldsymbol{V_k}$ is $\boldsymbol{T_k}$ and $\boldsymbol{V_{k+1}}$ is $\boldsymbol{T_{k+1}}$

#7 : Switching Sequence (7-segment) with Timing (Odd Sectors)

- Switching Sequence (7-segment) with Timing at sector k=1,3,5

Conventional 7-segment switching sequence (sector k=1,3,5)



Example for sector 3

In sector 3, where $k = 3$, therefore, $V_3(010)$ and $V_4(011)$

In a sampling period T_s

1: $time < \frac{T_0}{4} : V_0(000)$

2: $time < \frac{T_0}{4} + \frac{T_k}{2} : V_3(010)$

3: $time < \frac{T_0}{4} + \frac{T_k}{2} + \frac{T_{k+1}}{2} : V_4(011)$

4: $time < \frac{3\,T_0}{4} + \frac{T_k}{2} + \frac{T_{k+1}}{2} : V_7(111)$

5: $time < \frac{3\,T_0}{4} + \frac{T_k}{2} + T_{k+1} : V_4(011)$

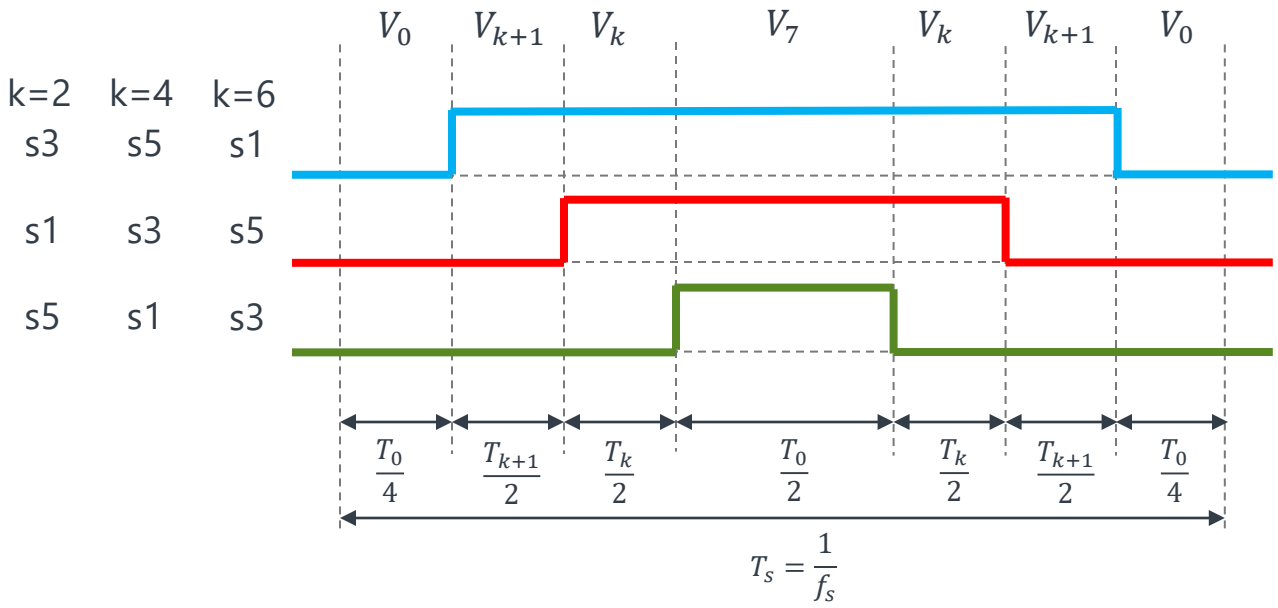
6: $time < \frac{3\,T_0}{4} + T_k + T_{k+1} : V_3(010)$

7: $time < T_s : V_0(000)$

#7 : Switching Sequence (7-segment) with Timing (Even Sectors)

- Switching Sequence (7-segment) with Timing at sector k=2,4,6

Conventional 7-segment switching sequence (sector k=2,4,6)



Example for sector 2
In sector 2, where $k = 2$, therefore, $V_2(110)$ and $V_3(010)$

In a sampling period T_s

1: $time < \frac{T_0}{4} : V_0(000)$

2: $time < \frac{T_0}{4} + \frac{T_{k+1}}{2} : V_3(010)$

3: $time < \frac{T_0}{4} + \frac{T_{k+1}}{2} + \frac{T_k}{2} : V_2(110)$

4: $time < \frac{3\,T_0}{4} + \frac{T_{k+1}}{2} + \frac{T_k}{2} : V_7(111)$

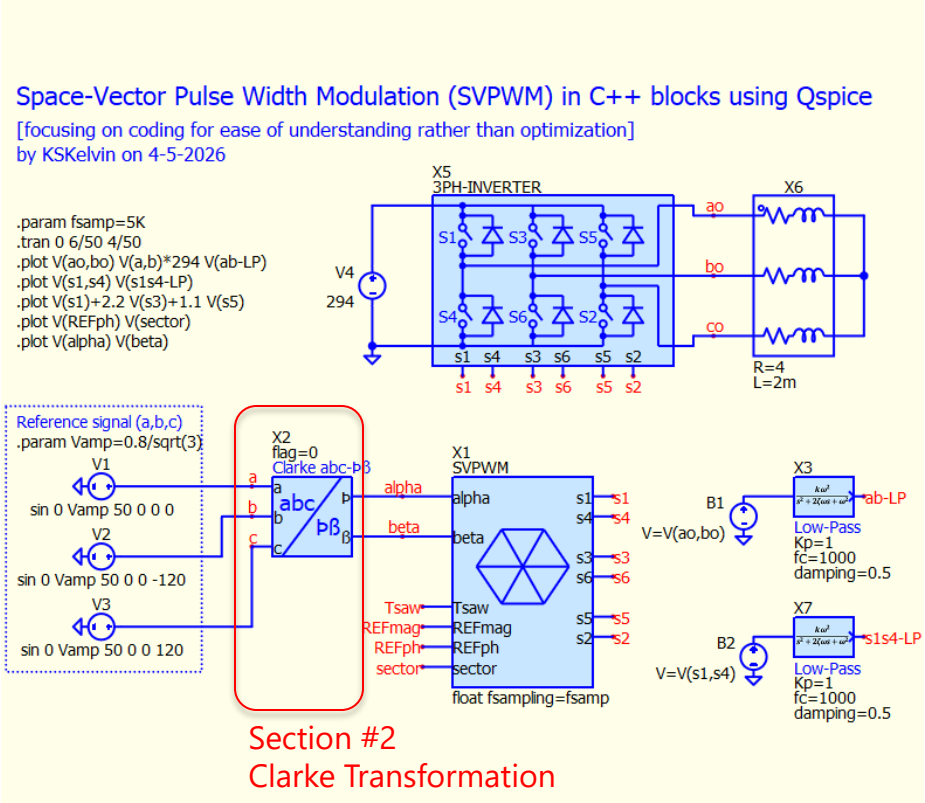
5: $time < \frac{3\,T_0}{4} + \frac{T_{k+1}}{2} + T_k : V_2(110)$

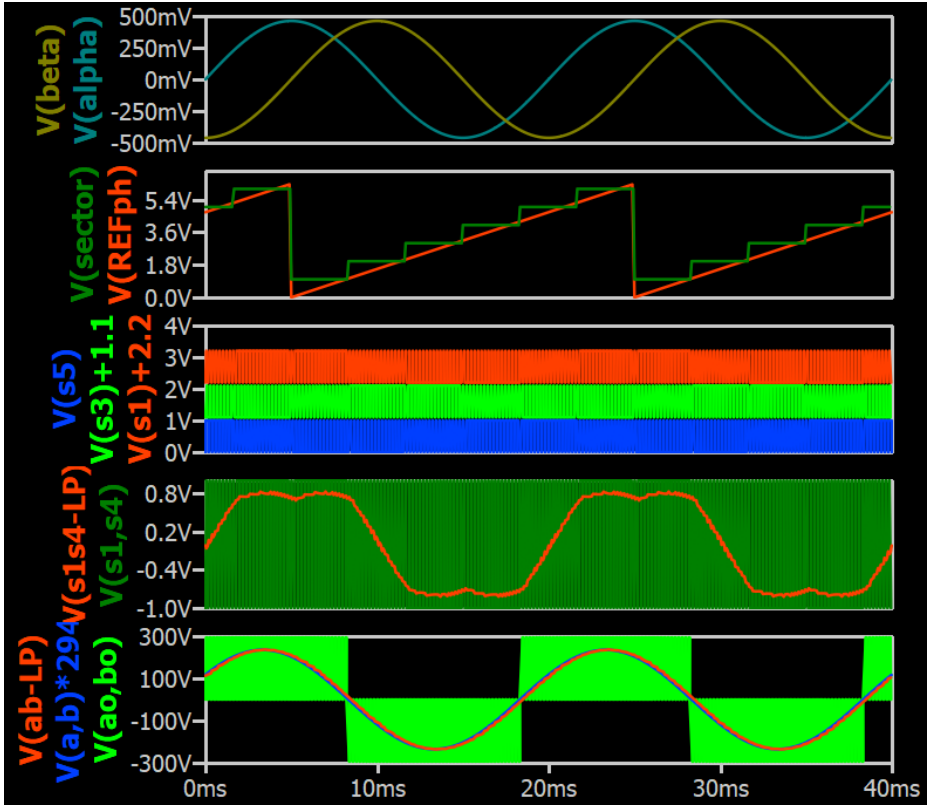
6: $time < \frac{3\,T_0}{4} + T_{k+1} + T_k : V_3(010)$

7: $time < T_s : V_0(000)$

#7: Switching Sequence – 7 segment scheme

Qspice : \02 Basic Operation (DLL 7-segment)\example.Space-VectorPWM(7-segment).qsch





#7: Switching Sequence – 7 segment scheme

Qspice : \02 Basic Operation (DLL 7-segment)\svpwm.cpp

Section #2 : Target Reference Vector

Section #4: Sector identification

Section #5: Timing calculation

```
// Calculate PWM period from sampling frequency
double Ts=1.0/fsampling;

// Calculate reference voltage vector magnitude and angle
REFmag = hypot(alpha,beta);          // Euclidean norm: sqrt(alpha² + beta²)
REFph = atan2(-beta,-alpha)+M_PI;    // Phase angle from Cartesian coordinates, [0, 2

// Determine which sector the reference vector falls into (6 sectors of 60°(pi/3) ea
sector = trunc(REFph/(M_PI/3))+1;

/// Generate triangular sawtooth waveform synchronized with sampling frequency
Tsaw = fmod(t,Ts);   // Modulo operation creates repeating ramp from 0 to Ts

// Calculate duty cycles for adjacent active vectors (Tk and Tk+1)
double Tk=0; double Tkplus1=0;
int method=1;  // Selects which formula to use (0: alpha-beta, 1: magnitude-phase, 2
// These represent the on-times for the two vectors bordering the current sector
// Tk = on-time for the first active vector (counter-clockwise from sector start)
// Tkplus1 = on-time for the second active vector (clockwise from sector end)
// All formulas produce equivalent results mathematically but differ in computationa
if(method==0){
   // Formula 1 : Based on alpha-beta (stationary reference frame) components
   Tk      = sqrt(3.0)*Ts*( sin(sector*M_PI/3.0)*alpha - cos(sector*M_PI/3.0)*beta )
   Tkplus1 = sqrt(3.0)*Ts*( -sin((sector-1)*M_PI/3.0)*alpha + cos((sector-1)*M_PI/3.
}else if(method==1){
   // Formula 2 : Based on reference voltage magnitude and absolute phase angle
   Tk      = sqrt(3.0)*REFmag*Ts*( sin(sector*M_PI/3.0)*cos(REFph) - cos(sector*M_PI
   Tkplus1 = sqrt(3.0)*REFmag*Ts*( -sin((sector-1)*M_PI/3.0)*cos(REFph) + cos((secto
}else if(method==2){
   // Formula 3 : Simplified angle-based calculation (most efficient)
   // Uses relative angle within the current sector (0° to 60°), with respect to Vk
   Tk      = sqrt(3.0)*REFmag*Ts*sin(M_PI/3-(REFph-trunc(REFph*3.0/M_PI)*M_PI/3.0));
   Tkplus1 = sqrt(3.0)*REFmag*Ts*sin((REFph-trunc(REFph*3.0/M_PI)*M_PI/3.0));
}
// Zero vector on-time (remaining time in switching period)
double T0      = Ts - (Tk + Tkplus1);
```

Section #7 : 7-segment Switching Sequence

```
// Seven-segment SVPWM pattern generation for each sector
// The switching sequence ensures minimum switching losses and reduces harmonics
if(sector == 1){
   // Sector 1 switching sequence:
   //  V0   ->  V1   ->  V2   ->  V7   ->  V2   ->  V1   ->   V0
   // (000) -> (100) -> (110) -> (111) -> (110) -> (100) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0))                    {s1=true ;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0))        {s1=true ;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1+T0/2.0))     {s1=true ;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tk+Tkplus1+T0/2.0))         {s1=true ;s3=false;s5=false;}
   else                                               {s1=false;s3=false;s5=false;}
}
if(sector == 2){
   // Sector 2 switching sequence (modified for single-bit transitions):
   //  V0   ->  V3   ->  V2   ->  V7   ->  V2   ->  V3   ->   V0
   // (000) -> (010) -> (110) -> (111) -> (110) -> (010) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0))               {s1=false;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0))        {s1=true ;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk+T0/2.0))     {s1=true ;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1+Tk+T0/2.0))         {s1=false;s3=true ;s5=false;}
   else                                               {s1=false;s3=false;s5=false;}
}
if(sector == 3){
   // Sector 3 switching sequence:
   //  V0   ->  V3   ->  V4   ->  V7   ->  V4   ->  V3   ->   V0
   // (000) -> (010) -> (011) -> (111) -> (011) -> (010) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0))                    {s1=false;s3=true ;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0))        {s1=false;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1+T0/2.0))     {s1=false;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk+Tkplus1+T0/2.0))         {s1=false;s3=true ;s5=false;}
   else                                               {s1=false;s3=false;s5=false;}
}
if(sector == 4){
   // Sector 4 switching sequence (modified for single-bit transitions):
   //  V0   ->  V5   ->  V4   ->  V7   ->  V4   ->  V5   ->   V0
   // (000) -> (001) -> (011) -> (111) -> (011) -> (001) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0))               {s1=false;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0))        {s1=false;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk+T0/2.0))     {s1=false;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1+Tk+T0/2.0))         {s1=false;s3=false;s5=true ;}
   else                                               {s1=false;s3=false;s5=false;}
}
if(sector == 5){
   // Sector 5 switching sequence:
   //  V0   ->  V5   ->  V6   ->  V7   ->  V6   ->  V5   ->   V0
   // (000) -> (001) -> (101) -> (111) -> (101) -> (001) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tk/2.0))                    {s1=false;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0))        {s1=true ;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk/2.0+Tkplus1+T0/2.0))     {s1=true ;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tk+Tkplus1+T0/2.0))         {s1=false;s3=false;s5=true ;}
   else                                               {s1=false;s3=false;s5=false;}
}
if(sector == 6){
   // Sector 6 switching sequence (modified for single-bit transitions):
   //  V0   ->  V1   ->  V6   ->  V7   ->  V6   ->  V1   ->   V0
   // (000) -> (100) -> (101) -> (111) -> (101) -> (100) -> (000)
   if(Tsaw < T0/4.0)                                  {s1=false;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0))               {s1=true ;s3=false;s5=false;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0))        {s1=true ;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk/2.0+T0/2.0)) {s1=true ;s3=true ;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1/2.0+Tk+T0/2.0))     {s1=true ;s3=false;s5=true ;}
   else if(Tsaw < (T0/4.0+Tkplus1+Tk+T0/2.0))         {s1=true ;s3=false;s5=false;}
   else                                               {s1=false;s3=false;s5=false;}
}
```

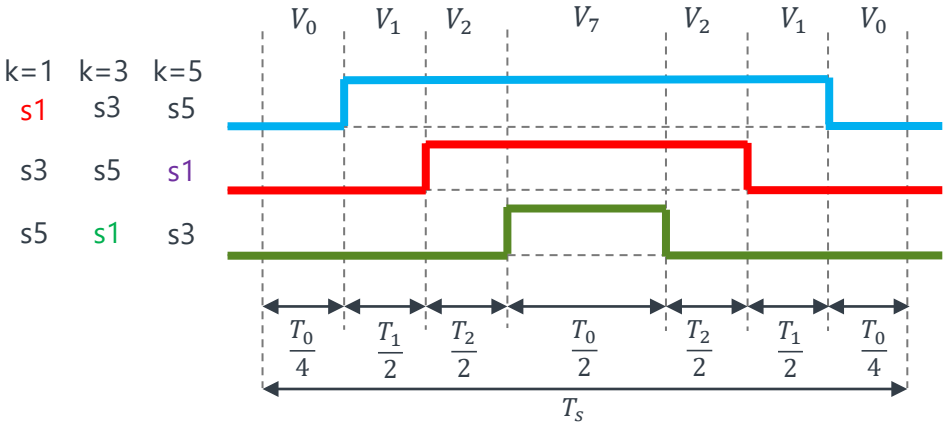
B-source SVPWM Implementation in Qspice

#8: B-source SVPWM implementation with Local Geometry

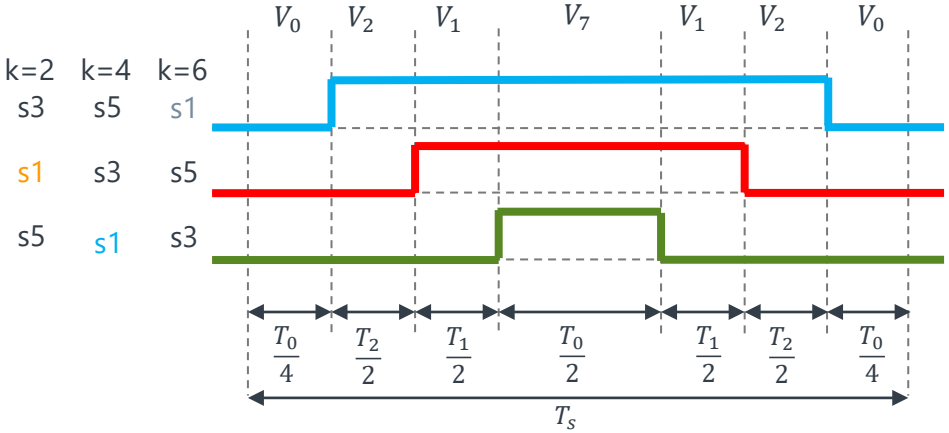
- General Form – Local geometry
  - Analog implementation requires $\vec{V}_k$-axis (sector bound local frame) timing (T_k and T_{k+1}) computation, and switching sequence implemented with a lookup table
  - Local geometry for all sector, where ϕ is angle between $\vec{V}_{ref}$ and $\vec{V}_k$ (where $0° \leq \phi < 60°$)
    - $T_k = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} \sin\left(\frac{\pi}{3} - \phi\right)$
    - $T_{k+1} = \frac{\sqrt{3}\,T_s|V_{ref}|}{V_{dc}} sin\phi$
    - where $\phi = \theta_{ref} - \mathrm{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) \cdot \frac{\pi}{3}$
  - By nominalize $V_{dc} = 1$ and the sawtooth in comparison with magnitude between 0 to 1 every T_s period (i.e. also nominalize $T_S = 1$), general form can further simplified to,
    - $T_1 = \sqrt{3}\,|V_{ref}| \sin\left(\frac{\pi}{3} - \phi\right)$
    - $T_2 = \sqrt{3}\,|V_{ref}| \sin\phi$
    - where $\phi = \theta_{ref} - \mathrm{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) \cdot \frac{\pi}{3}$
    - $T_0 = 1 - T_1 - T_2$

Switching Pattern of Local Geometry

Conventional 7-segment switching sequence (sector k=1,3,5)



Conventional 7-segment switching sequence (sector k=2,4,6)



In this section, local timing within the sector is explained, detailing how the local switching pattern is derived from the 7-segment scheme

For example, s1 on time can be summarized as

Sector 1 (s1 @ k=1) : On time = $\frac{T_0}{2} + T_1 + T_2$

Sector 2 (s1 @ k=2) : On time = $\frac{T_0}{2} + T_1$

Sector 3 (s1 @ k=3) : On time = $\frac{T_0}{2}$

Sector 4 (s1 @ k=4) : On time = $\frac{T_0}{2}$

Sector 5 (s1 @ k=5) : On time = $\frac{T_0}{2} + T_2$

Sector 6 (s1 @ k=6) : On time = $\frac{T_0}{2} + T_1 + T_2$

With this method, now can derive off time or on time for s1, s3 and s5 in next slide

#9a : General Form of 7-segment algorithm (Local Geometry)

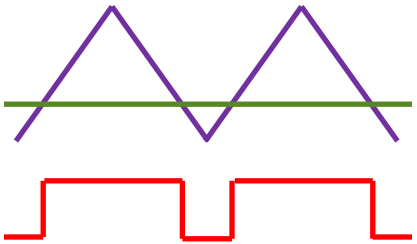
ON Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2}+T_2$	$\frac{T_0}{2}+T_1+T_2$
s3	$\frac{T_0}{2}+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$
s5	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2}+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1$

OFF Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	$\frac{T_0}{2}$	$\frac{T_0}{2}+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1$	$\frac{T_0}{2}$
s3	$\frac{T_0}{2}+T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2}+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1+T_2$
s5	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1+T_2$	$\frac{T_0}{2}+T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2}+T_2$

Comparator

IN+ : On time level

IN- : Sawtooth



Generate s1, s3 and s5 compare threshold and gating pattern

A comparator is necessary to produce the gating pattern for s1, s3, and s5. By comparing the on-time thresholds of s1, s3, and s5 with a symmetric triangular sawtooth signal at IN-, the 7-segment switching pattern can be generated. This results in the same gating pattern discussed in the previous section. The timing thresholds for s1, s3, and s5 are calculated based on the on-time in lookup table corresponding to the sector of the reference voltage vector.

B-source SVPWM implementation in Qspice

Qspice : \03a B-source SVPWM Implementation (7-segment)\SVPWM.qsch

$$T_1 = \sqrt{3}\,|V_{ref}|\sin\left(\frac{\pi}{3} - \phi\right)$$

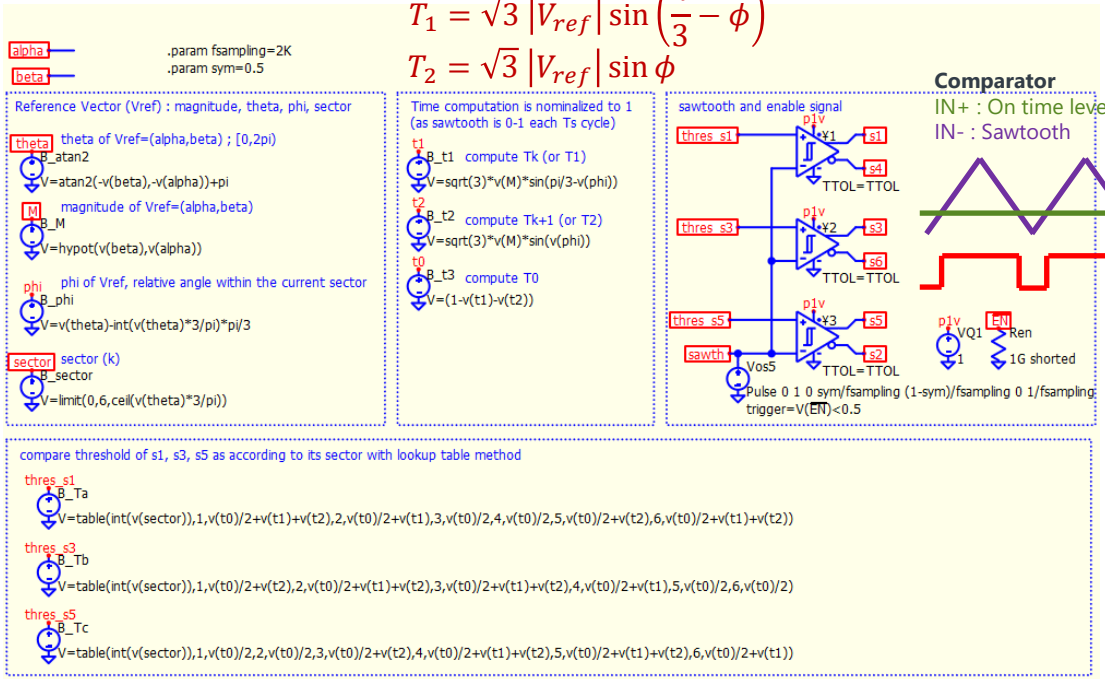
$$T_2 = \sqrt{3}\,|V_{ref}|\sin\phi$$

$$\theta_{ref} = \tan^{-1}\left(\frac{\beta}{\alpha}\right)$$

$$|V_{ref}| = \sqrt{\alpha^2 + \beta^2}$$

$$\phi = \theta_{ref} - \mathrm{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right)\cdot\frac{\pi}{3}$$

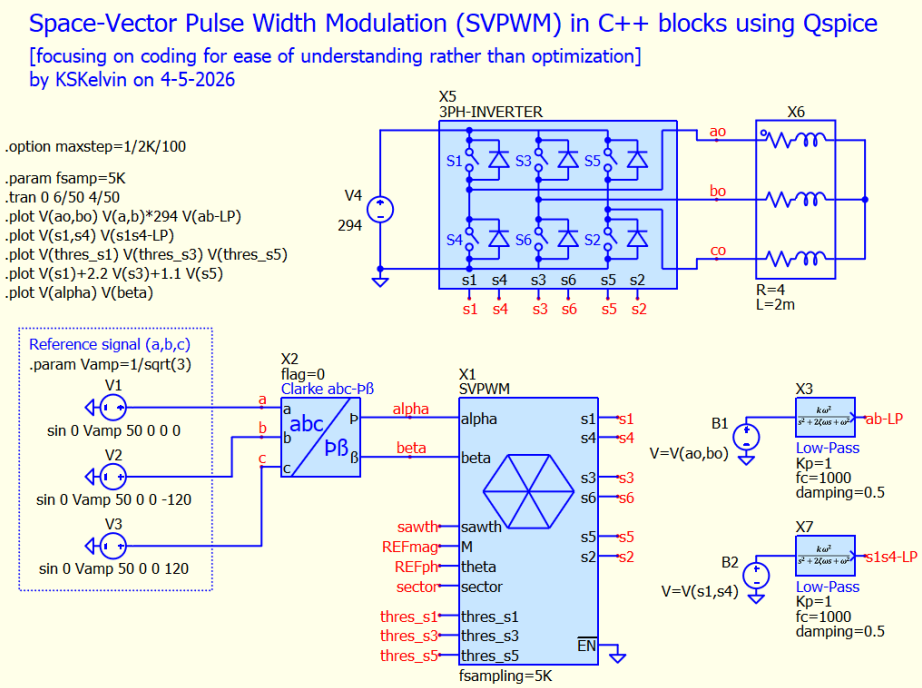
$$sector(k) = \mathrm{trunc}\left(\frac{\theta_{ref}}{\frac{\pi}{3}}\right) + 1$$

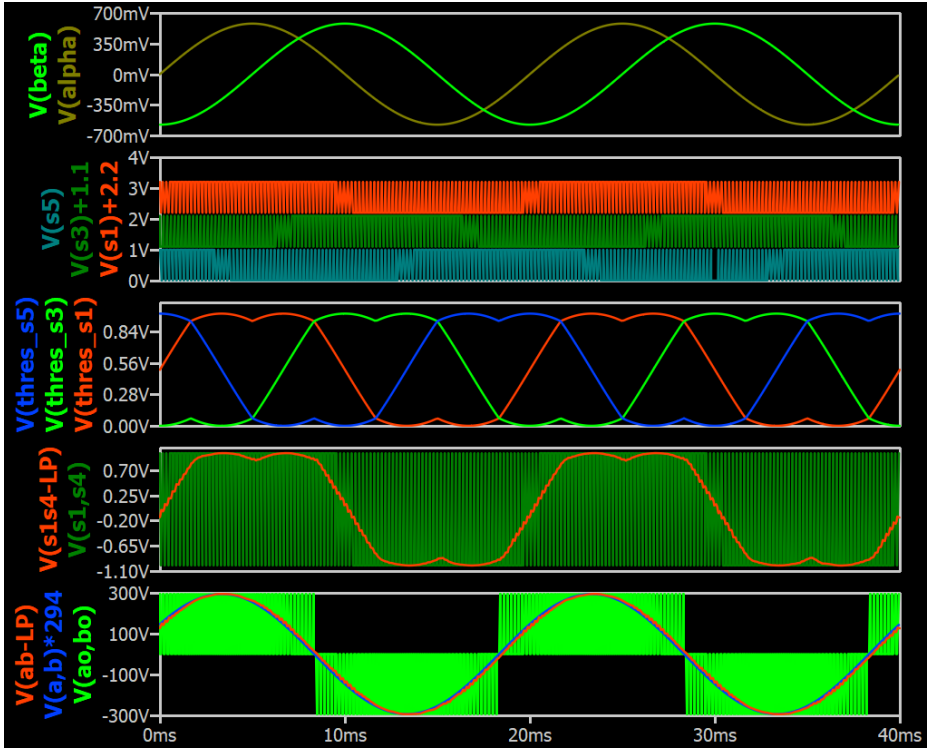


ON Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	$\frac{T_0}{2} + T_1 + T_2$	$\frac{T_0}{2} + T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2} + T_2$	$\frac{T_0}{2} + T_1 + T_2$
s3	$\frac{T_0}{2} + T_2$	$\frac{T_0}{2} + T_1 + T_2$	$\frac{T_0}{2} + T_1 + T_2$	$\frac{T_0}{2} + T_1$	$\frac{T_0}{2}$	$\frac{T_0}{2}$
s5	$\frac{T_0}{2}$	$\frac{T_0}{2}$	$\frac{T_0}{2} + T_2$	$\frac{T_0}{2} + T_1 + T_2$	$\frac{T_0}{2} + T_1 + T_2$	$\frac{T_0}{2} + T_1$

B-source SVPWM implementation in Qspice

Qspice : \03 B-source SVPWM Implementation\example.Space-VectorPWM(B-source).qsch





B-source SVPWM implementation in Qspice

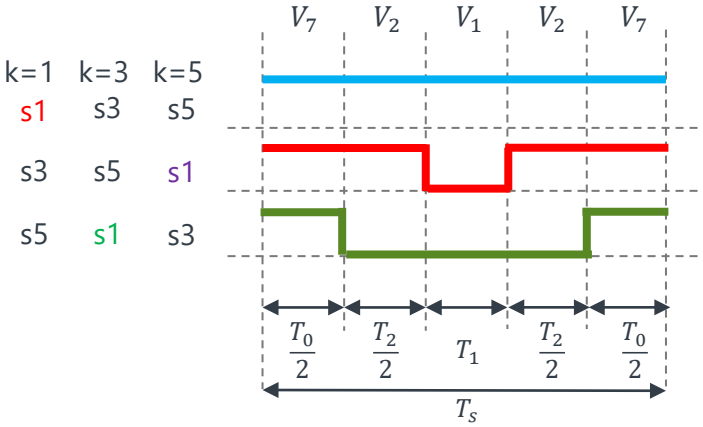
- Summary
  - Timing calculation and Switching Pattern in Local Geometry with respect to $\vec{V}_k$-axis (sector bound local frame) is essential in simplifying computing of SVPWM for real time control
  - Instead of treating the inverter as a system with 6 unique sectors, local geometry treats it as **1 universal sector** that simply “moves” around
  - With local geometry
    - The Equations are Identical
    - The Logic is Reusable

Switching Pattern
5-segment Algorithm
(with Local Geometry)

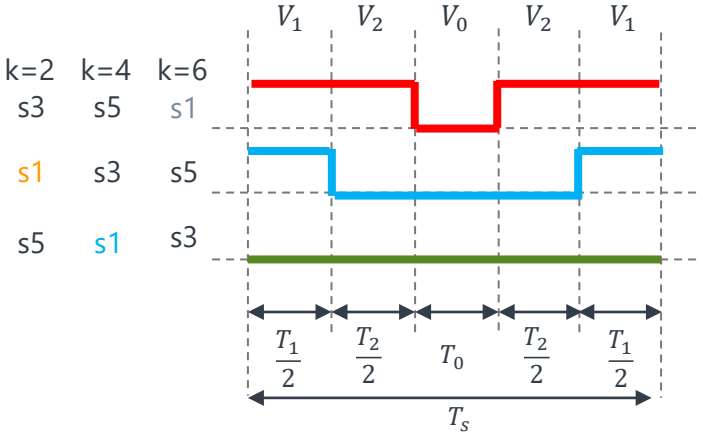
Switching Sequence of 5-segment algorithm

- Switching Sequence of 5-segment algorithm
  - This 5-segment switching sequence follows a specific pattern with only **single-bit changes** to minimize switching actions during transitions
  - Sector 1 : $V7(111) \rightarrow V2(110) \rightarrow V1(100) \rightarrow V2(110) \rightarrow V7(111)$
  - Sector 2 : $V2(110) \rightarrow V3(010) \rightarrow V0(000) \rightarrow V3(010) \rightarrow V2(110)$
  - Sector 3 : $V7(111) \rightarrow V4(011) \rightarrow V3(010) \rightarrow V4(011) \rightarrow V7(111)$
  - Sector 4 : $V4(011) \rightarrow V5(001) \rightarrow V0(000) \rightarrow V5(001) \rightarrow V4(011)$
  - Sector 5 : $V7(111) \rightarrow V6(101) \rightarrow V5(001) \rightarrow V6(101) \rightarrow V7(111)$
  - Sector 6 : $V6(101) \rightarrow V1(100) \rightarrow V0(000) \rightarrow V1(100) \rightarrow V6(101)$
  - 5-segment PWM reduce switching frequency as compare to 7-segment PWM for higher efficiency at high switching frequencies or high power applications with higher total harmonic distortion (THD) trade-off
  - As a result, switching sequence are
    - In odd sector ($\boldsymbol{k = 1, 3, 5}$): $\boldsymbol{V_7 \rightarrow V_{k+1} \rightarrow V_k \rightarrow V_{k+1} \rightarrow V_7}$
    - In even sector ($\boldsymbol{k = 2, 4, 6}$): $\boldsymbol{V_k \rightarrow V_{k+1} \rightarrow V_0 \rightarrow V_{k+1} \rightarrow V_k}$
    - where timing of $\boldsymbol{V_0}$ / $\boldsymbol{V_7}$ is $\boldsymbol{T_0}$, $\boldsymbol{V_k}$ is $\boldsymbol{T_k}$ and $\boldsymbol{V_{k+1}}$ is $\boldsymbol{T_{k+1}}$

Conventional 5-segment switching sequence (sector k=1,3,5)



Conventional 5-segment switching sequence (sector k=2,4,6)



General Form of Switching Pattern

For example, s1 on time can be summarized as

Sector 1 (s1 @ k=1) : On time = $T_0 + T_1 + T_2$
Sector 2 (s1 @ k=2) : On time = T_1
Sector 3 (s1 @ k=3) : On time = T_0
Sector 4 (s1 @ k=4) : On time = 0
Sector 5 (s1 @ k=5) : On time = $T_0 + T_2$
Sector 6 (s1 @ k=6) : On time = $T_1 + T_2$

Sector 1 (s3 @ k=1) : On time = $T_0 + T_2$
Sector 2 (s3 @ k=2) : On time = $T_1 + T_2$
Sector 3 (s3 @ k=3) : On time = $T_0 + T_1 + T_2$
Sector 4 (s3 @ k=4) : On time = T_1
Sector 5 (s3 @ k=5) : On time = T_0
Sector 6 (s3 @ k=6) : On time = 0

Sector 1 (s5 @ k=1) : On time = T_0
Sector 2 (s5 @ k=2) : On time = 0
Sector 3 (s5 @ k=3) : On time = $T_0 + T_2$
Sector 4 (s5 @ k=4) : On time = $T_1 + T_2$
Sector 5 (s5 @ k=5) : On time = $T_0 + T_1 + T_2$
Sector 6 (s5 @ k=6) : On time = T_1

#9b: General Form of 5-segment algorithm (Local Geometry)

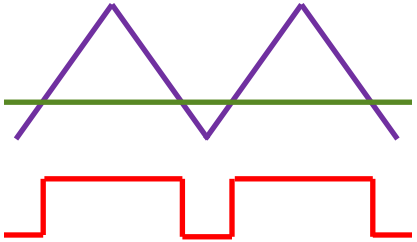
ON Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	$T_0 + T_1 + T_2$	T_1	T_0	0	$T_0 + T_2$	$T_1 + T_2$
s3	$T_0 + T_2$	$T_1 + T_2$	$T_0 + T_1 + T_2$	T_1	T_0	0
s5	T_0	0	$T_0 + T_2$	$T_1 + T_2$	$T_0 + T_1 + T_2$	T_1

OFF Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	0	$T_0 + T_2$	$T_1 + T_2$	$T_0 + T_1 + T_2$	T_1	T_0
s3	T_1	T_0	0	$T_0 + T_2$	$T_1 + T_2$	$T_0 + T_1 + T_2$
s5	$T_1 + T_2$	$T_0 + T_1 + T_2$	T_1	T_0	0	$T_0 + T_2$

Comparator

IN+ : On time level

IN- : Sawtooth

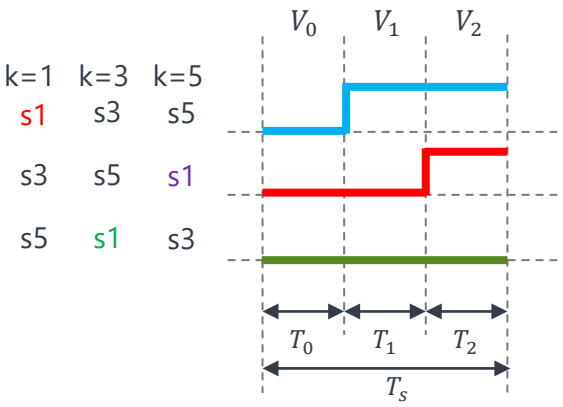


Switching Pattern 3-segment Algorithm (with Local Geometry)

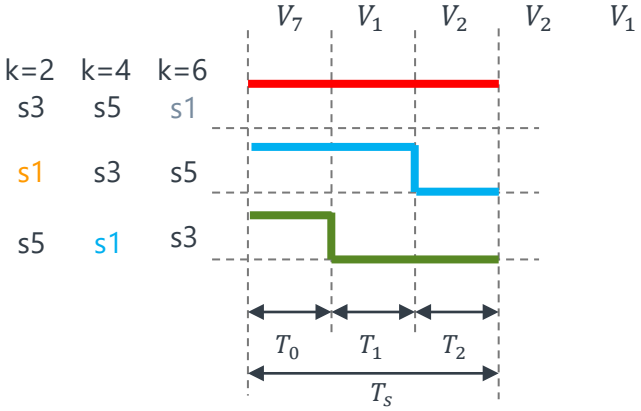
Switching Sequence of 3-segment algorithm

- 3-Segment Switching Sequence
  - This 3-segment switching sequence follows a specific pattern with only **single-bit changes** to minimize switching actions during transitions
  - Sector 1 : $V_0(000) \rightarrow V_1(100) \rightarrow V_2(110)$
  - Sector 2 : $V_7(111) \rightarrow V_2(110) \rightarrow V_3(010)$
  - Sector 3 : $V_0(000) \rightarrow V_3(010) \rightarrow V_4(011)$
  - Sector 4 : $V_7(111) \rightarrow V_4(011) \rightarrow V_5(001)$
  - Sector 5 : $V_0(000) \rightarrow V_5(001) \rightarrow V_6(101)$
  - Sector 6 : $V_7(111) \rightarrow V_6(101) \rightarrow V_1(100)$
  - As a result, switching sequence are
    - In odd sector ($\boldsymbol{k = 1, 3, 5}$): $\boldsymbol{V_0 \rightarrow V_k \rightarrow V_{k+1}}$
    - In even sector ($\boldsymbol{k = 2, 4, 6}$): $\boldsymbol{V_7 \rightarrow V_k \rightarrow V_{k+1}}$
    - where timing of $\boldsymbol{V_0}$ / $\boldsymbol{V_7}$ is $\boldsymbol{T_0}$, $\boldsymbol{V_k}$ is $\boldsymbol{T_k}$ and $\boldsymbol{V_{k+1}}$ is $\boldsymbol{T_{k+1}}$

Conventional 3-segment switching sequence (sector k=1,3,5)



Conventional 3-segment switching sequence (sector k=2,4,6)



General Form of Switching Pattern

For example, s1 on time can be summarized as

Sector 1 (s1 @ k=1) : On time = $T_1 + T_2$
Sector 2 (s1 @ k=2) : On time = $T_0 + T_1$
Sector 3 (s1 @ k=3) : On time = 0
Sector 4 (s1 @ k=4) : On time = T_0
Sector 5 (s1 @ k=5) : On time = T_2
Sector 6 (s1 @ k=6) : On time = $T_0 + T_1 + T_2$

Sector 1 (s3 @ k=1) : On time = T_2
Sector 2 (s3 @ k=2) : On time = $T_0 + T_1 + T_2$
Sector 3 (s3 @ k=3) : On time = $T_1 + T_2$
Sector 4 (s3 @ k=4) : On time = $T_0 + T_1$
Sector 5 (s3 @ k=5) : On time = 0
Sector 6 (s3 @ k=6) : On time = T_0

Sector 1 (s5 @ k=1) : On time = 0
Sector 2 (s5 @ k=2) : On time = T_0
Sector 3 (s5 @ k=3) : On time = T_2
Sector 4 (s5 @ k=4) : On time = $T_0 + T_1 + T_2$
Sector 5 (s5 @ k=5) : On time = $T_1 + T_2$
Sector 6 (s5 @ k=6) : On time = $T_0 + T_1$

#9c : General Form of 3-segment algorithm (Local Geometry)

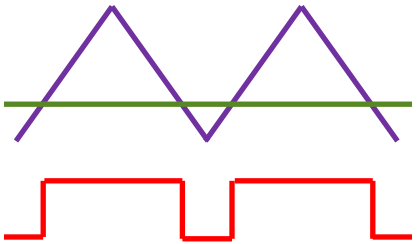
ON Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	$T_1 + T_2$	$T_0 + T_1$	0	T_0	T_2	$T_0 + T_1 + T_2$
s3	T_2	$T_0 + T_1 + T_2$	$T_1 + T_2$	$T_0 + T_1$	0	T_0
s5	0	T_0	T_2	$T_0 + T_1 + T_2$	$T_1 + T_2$	$T_0 + T_1$

OFF Time	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6
s1	T_0	T_2	$T_0 + T_1 + T_2$	$T_1 + T_2$	$T_0 + T_1$	0
s3	$T_0 + T_1$	0	T_0	T_2	$T_0 + T_1 + T_2$	$T_1 + T_2$
s5	$T_0 + T_1 + T_2$	$T_1 + T_2$	$T_0 + T_1$	0	T_0	T_2

Comparator

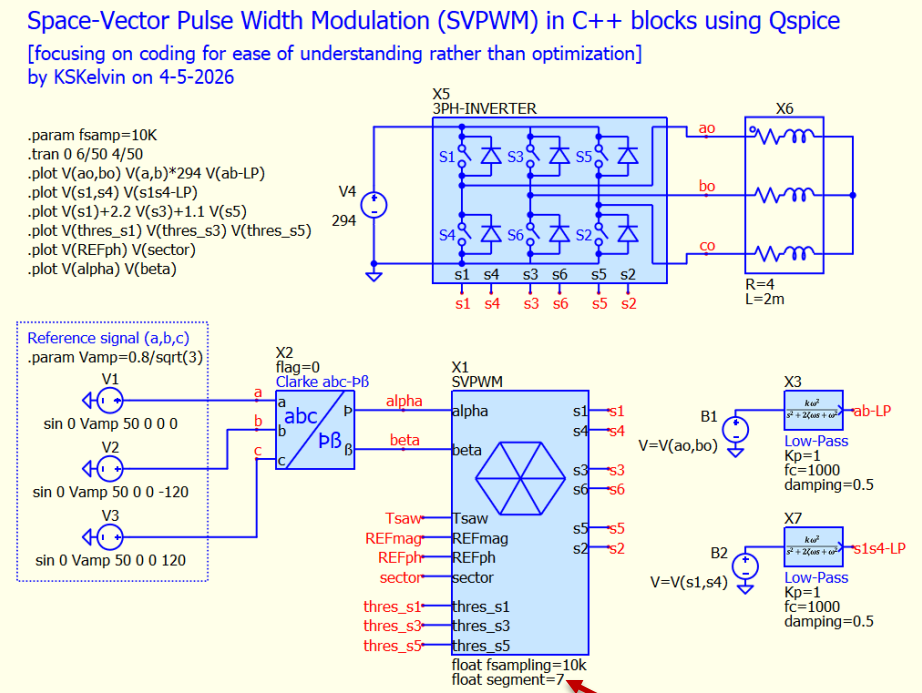
IN+ : On time level

IN- : Sawtooth

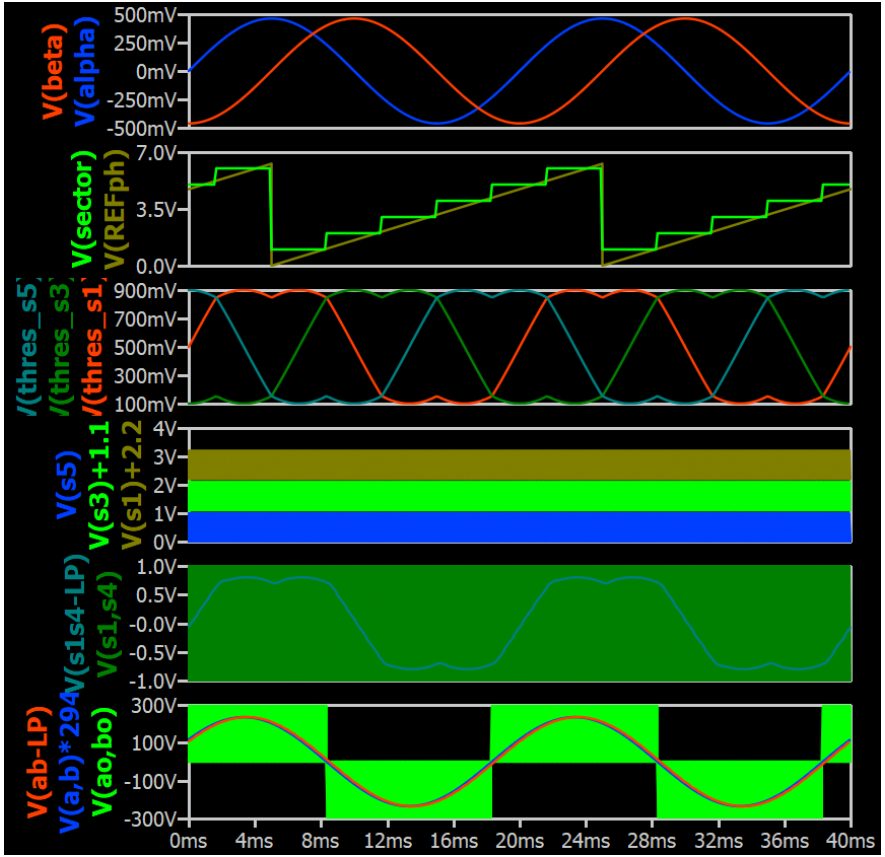


SVPWM with Local Geometry Implementation (C++ version)

Qspice : \04 Local Geometry (DLL)\example.Space-VectorPWM(LocalGeometry).qsch



Segment = 3, 5 or 7



SVPWM with Local Geometry Implementation (C++ version)

Qspice : \04 Local Geometry (DLL)\svpwm.cpp

```
// Calculate PWM period from sampling frequency
double Ts=1.0/fsampling;

// Calculate reference voltage vector magnitude and angle
REFmag = hypot(alpha,beta);          // Euclidean norm: sqrt(alpha
REFph = atan2(-beta,-alpha)+M_PI;    // Phase angle from Cartesian

// Determine which sector the reference vector falls into (6 sect
sector = trunc(REFph/(M_PI/3))+1;

// Generate normalized symmetric triangular carrier waveform (ran
// Creates triangle wave that rises from 0 to 1 over Ts/2, then f
sawth = (fabs(fmod(t, Ts) - Ts/2.0) * 2.0)/Ts;   // Normalized sa

// Calculate timing for T1 and T2
double phi = REFph-trunc(REFph*3.0/M_PI)*M_PI/3.0;
// Uses relative angle within the current sector (0° to 60°), wit
// Values are normalized such that T1+T2+T0 = 1 (where Ts=1)
double T1 = sqrt(3.0)*REFmag*sin(M_PI/3-phi);
double T2 = sqrt(3.0)*REFmag*sin(phi);
// Zero vector on-time (remaining time in switching period) [norm
double T0 = 1 - (T1 + T2);
```

Symmetric Sawtooth (comparator)

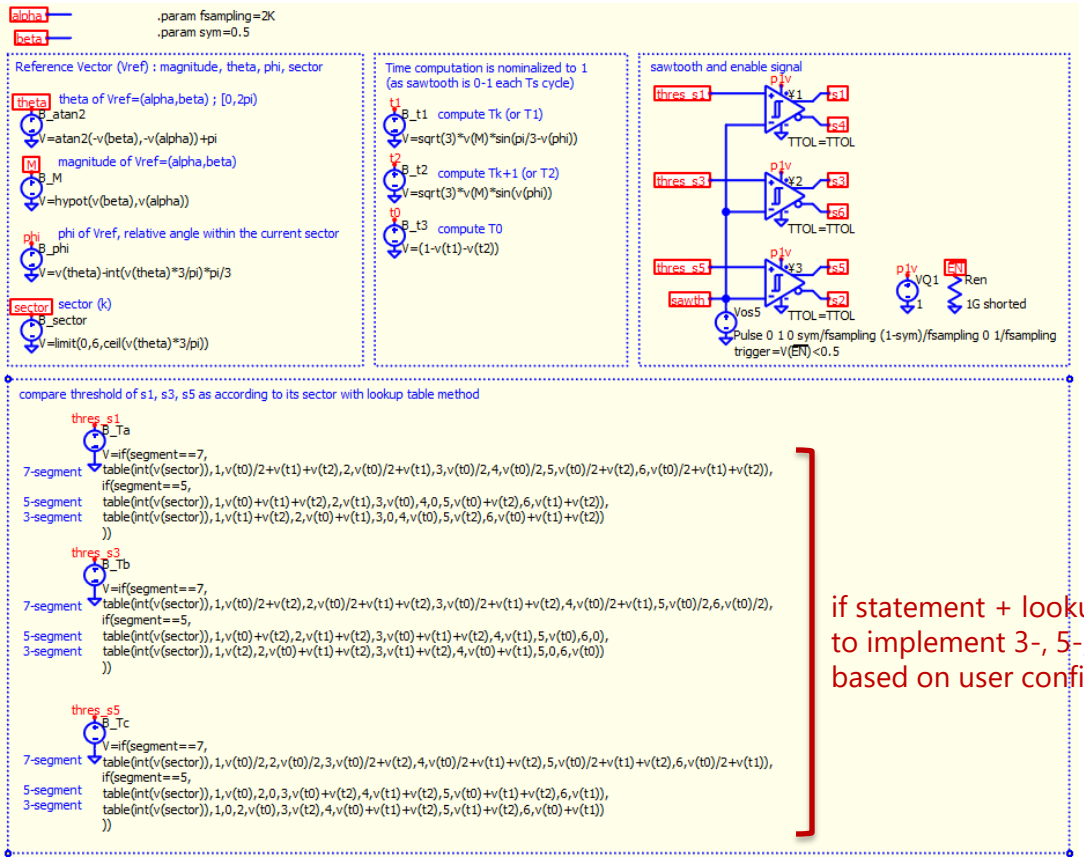
Section #8: Timing calculation (Local geometry)

```
// Calculate PWM threshold values for each phase based on sector an
// These thresholds determine when each upper switch (s1, s3, s5) t
// Different thresholds create the required switching sequence for

// 3-segment switching pattern with local geometry
if(segment==3){
   if(sector==1){
      thres_s1 = T1+T2;
      thres_s3 = T2;
      thres_s5 = 0;
   }else if(sector==2){
      thres_s1 = T0+T1;
      thres_s3 = T0+T1+T2;
      thres_s5 = T0;
   }else if(sector==3){
      thres_s1 = 0;
      thres_s3 = T1+T2;
      thres_s5 = T2;
   }else if(sector==4){
      thres_s1 = T0;
      thres_s3 = T0+T1;

         :

      thres_s3 = T0/2+T1;
      thres_s5 = T0/2+T1+T2;
   }else if(sector==5){
      thres_s1 = T0/2+T2;
      thres_s3 = T0/2;
      thres_s5 = T0/2+T1+T2;
   }else{
      thres_s1 = T0/2+T1+T2;
      thres_s3 = T0/2;
      thres_s5 = T0/2+T1;
   }
}

// Comparator: Generate PWM signals by comparing normalized t
// Switch turns ON when carrier waveform (sawth) is BELOW the
if(thres_s1 > sawth) s1=true; else s1 = false;
if(thres_s3 > sawth) s3=true; else s3 = false;
if(thres_s5 > sawth) s5=true; else s5 = false;
```

Section #9a, 9b, 9c: Switching pattern Select from segment 3, 5 or 7 (Local geometry)

SVPWM with Local Geometry Implementation (B-source Version)

Qspice : \03b B-source SVPWM Implementation (3-5-7-segment)\SVPWM.qsch



if statement + lookup table
to implement 3-, 5-, 7-segment
based on user configuration